

Complex SQL Subquery Assignment — Questions with Expectations

Use the SmartMart database tables:

- stores
- employees
- customers
- categories
- products
- orders
- order_items
- payments

Do not use correlated subqueries in any question.

1. Products Priced Above the Overall Average

Question

Find all products whose price is greater than the average price of all products.

Expected output

- Product ID
- Product name
- Category name
- Product price
- Overall average product price
- Difference from average price

Expectations

- Use a scalar subquery to calculate the overall average price.
 - Join `products` with `categories`.
 - Round calculated values to two decimal places.
 - Sort products by price in descending order.
-

2. Customers Spending Above Average

Question

Find customers whose total spending on completed orders is greater than the average spending of customers who have completed orders.

Expected output

- Customer ID
- Customer name
- Customer type
- Number of completed orders
- Total amount spent
- Average customer spending

Expectations

- Consider only completed orders.
 - Calculate sales after applying item-level discounts.
 - First calculate total spending customer-wise in a subquery.
 - Calculate the average of those customer totals in another subquery.
 - Do not use a correlated subquery.
 - Sort customers by total spending in descending order.
-

3. Second-Highest Selling Product

Question

Find the product or products having the second-highest total quantity sold.

Expected output

- Product ID
- Product name
- Category name
- Total quantity sold
- Total sales amount

Expectations

- Calculate total sold quantity product-wise.
- Use nested aggregate subqueries.
- Do not use:
- `LIMIT`
- `OFFSET`

- Window functions
 - Display all products tied at the second-highest quantity.
 - Calculate sales after applying discounts.
-

4. Employees Earning Above Company Average

Question

Find employees whose salary is greater than the average salary of all employees in the company.

Expected output

- Employee ID
- Employee name
- Designation
- Store name
- Employee salary
- Company average salary
- Difference from company average

Expectations

- Use a scalar subquery to calculate the company average salary.
 - Join `employees` with `stores`.
 - Do not calculate separate averages for each store.
 - Round calculated values to two decimal places.
 - Sort employees by salary in descending order.
-

5. Customers Who Purchased All Available Accessory Products

Question

Find customers whose number of distinct purchased accessory products is equal to the total number of products available in the `Accessories` category.

Expected output

- Customer ID
- Customer name
- Distinct accessory products purchased
- Total accessory products available

Expectations

- Consider only completed orders.

- Use one subquery to calculate the total number of accessory products.
 - Use another grouped subquery to calculate accessory products purchased customer-wise.
 - Compare the two calculated values.
 - Do not use a correlated subquery.
 - Count distinct product IDs.
-

6. Stores Performing Above Average

Question

Find stores whose completed-order sales are greater than the average completed-order sales of all stores.

Expected output

- Store ID
- Store name
- City
- Number of completed orders
- Total sales
- Average store sales
- Difference from average store sales

Expectations

- Calculate completed-order sales store-wise in a derived table.
 - Calculate the average of store totals using another subquery.
 - Apply item-level discounts.
 - Include only stores having completed orders.
 - Do not use a correlated subquery.
 - Sort stores by total sales in descending order.
-

7. Products Never Purchased by Premium Customers

Question

Find products whose IDs do not appear in completed orders placed by Premium customers.

Expected output

- Product ID
- Product name
- Category name
- Product price
- Stock quantity

Expectations

- Use a non-correlated `NOT IN` subquery.
 - The subquery must return product IDs purchased by Premium customers.
 - Consider only completed orders.
 - Ensure the subquery does not return `NULL`.
 - Include products never ordered by any customer.
 - Sort category-wise and product-wise.
-

8. Orders Above the Overall Average Order Value

Question

Find orders whose calculated total amount is greater than the average value of all orders.

Expected output

- Order ID
- Customer name
- Order timestamp
- Order status
- Order amount
- Overall average order amount
- Difference from average

Expectations

- Calculate each order amount in a grouped subquery.
 - Calculate the average order amount from the grouped results.
 - Apply item-level discounts.
 - Do not use a correlated subquery.
 - Round monetary values to two decimal places.
 - Sort by difference from average in descending order.
-

9. Managers Whose Team Sales Exceed Average Team Sales

Question

Find managers whose direct-report employees generated more completed-order sales than the average sales of all manager teams.

Expected output

- Manager ID
- Manager name

- Number of direct-report employees
- Number of completed team orders
- Total team sales
- Average team sales
- Difference from average team sales

Expectations

- Use the self-referencing `manager_id` relationship.
 - Calculate manager-wise team sales in a derived table.
 - Calculate average team sales from that derived result.
 - Consider only direct-report employees.
 - Consider only completed orders.
 - Apply item-level discounts.
 - Do not include a manager's personal sales in their team sales.
 - Do not use a correlated subquery.
-

10. Customers With Spending in the Top Three Customer Totals

Question

Find customers whose completed-order spending is among the three highest distinct customer spending amounts.

Expected output

- Customer ID
- Customer name
- Customer type
- Number of completed orders
- Total amount spent
- Spending rank category

The rank category should be:

- Highest Spender
- Second-Highest Spender
- Third-Highest Spender

Expectations

- Calculate total completed-order spending customer-wise.
- Use nested subqueries to identify the three highest distinct spending amounts.
- Do not use:
- Window functions
- `LIMIT`
- `OFFSET`

- Correlated subqueries
- Apply item-level discounts.
- Handle ties by displaying every customer having the same qualifying spending amount.
- Sort customers by total spending in descending order.